Государственное бюджетное профессиональное образовательное учреждение
Республики Хакасия «Хакасский политехнический колледж»
Специальность 09.02.07
Информационные системы и программирование
квалификация «Программист»

КУРСОВАЯ РАБОТА

ПО ПМ.01 «Разработка модулей программного обеспечения для компьютерных систем»

ТЕМА: РАЗРАБОТКА ПРИЛОЖЕНИЯ ДЛЯ УЧЕТА ЗАЯВОК ПО НЕИСПРАВНОСТЯМ ТЕХНИКИ В ГБУЗ РХ «АМКБ»

Руководитель:
_____ Д.С. Черкашин
(подпись)
_____
(оценка, дата)
Выполнил:
Студент группы ИС(ПРО) – 41
_____ А. А. Мальцева
(подпись)
_____.
(дата)

Абакан 2025 г

Государственное бюджетное профессиональное образовательное учреждение
Республики Хакасия «Хакасский политехнический колледж»
Специальность 09.02.07
Информационные системы и программирование
квалификация «Программист»

Утверждаю:
Председатель П(Ц)К:
_____ Гартман Е.П.
(Подпись) (ФИО)

ЗАДАНИЕ
Для выполнения курсовой работы

По ПМ 01 Разработка модулей программного обеспечения для компьютерных систем
Студента Мальцевой Анастасии Александровны
4 курса, группы ИС(ПРО)-41

1. Тема курсовой работы: «Разработка приложения для учета заявок по неисправностям техники в ГБУЗ РХ «АМКБ»
2. Структура и содержание курсовой работы.

Введение
1 Общая часть
1.1 Описание проектируемой предметной области
1.2 Анализ существующих решений
2 Анализ и проектирование
2.1 Проектирование функциональной архитектуры программного средства
2.2 Проектирование структуры базы данных
2.3 Формирование спецификации требований программного средства
3 Разработка программного модуля
3.1 Разработка объектов базы данных в выбранной СУБД
3.2 Разработка функциональной архитектуры программного средства
4 Тестирование программного модуля
4.1 Разработка тест-кейсов
4.2 Проведение модульного тестирования
Заключение
Список использованных источников
Приложения
Ссылка на систему контроля версий

Руководитель _____ Черкашин Д.С.
(подпись) (ФИО)

Студент _____ Мальцева А.А.
(подпись) (ФИО)

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 4
1 ОБЩАЯ ЧАСТЬ 6
1.1 Описание проектируемой предметной области 6
1.2 Анализ существующих решений 7
2 АНАЛИЗ И ПРОЕКТИРОВАНИЕ 11
2.1 Проектирование функциональной архитектуры программного средства 11
2.2 Проектирование структуры базы данных 17
2.3 Формирование спецификации требований программного средства 18
3 РАЗРАБОТКА ПРОГРАММНОГО МОДУЛЯ 21
3.1 Разработка объектов базы данных в выбранной СУБД 21
3.2 Разработка функциональной архитектуры программного средства 23
4 ТЕСТИРОВАНИЕ ПРОГРАММНОГО МОДУЛЯ 32
4.1 Разработка тест-кейсов 32
4.2 Проведение модульного тестирования 35
ЗАКЛЮЧЕНИЕ 42
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 43
ПРИЛОЖЕНИЯ 44
Ссылка на систему контроля версий 44

ВВЕДЕНИЕ

Данная курсовая работа является завершающим этапом МДК.01.01 «Разработка модулей программного обеспечения для компьютерных систем».

Объект исследования: предметная область «Разработка приложения для учета заявок по неисправностям техники в ГБУЗ РХ «АМКБ»».

Предмет исследования: разработка объекта исследования ««Разработка приложения для учета заявок по неисправностям техники в ГБУЗ РХ «АМКБ»».

Цель курсовой работы— разработка и поддержка программного обеспечения учета заявок по неисправностям техники в ГБУЗ РХ «АМКБ».

Задачи:

- Спроектировать функциональную архитектуру программного средства;
- Спроектировать структуру базы данных;
- Реализовать функционал архитектуры программного средства;
- Провести модульное тестирование.

В условиях цифровой трансформации современных предприятий и организаций эффективное управление внутренними процессами, включая техническое обслуживание и ремонт оборудования, становится ключевым фактором обеспечения непрерывности бизнес-процессов. Особенно остро эта задача стоит в компаниях, где парк техники значителен, а своевременное устранение неисправностей напрямую влияет на производительность труда сотрудников и качество предоставляемых услуг.

Курсовая работа состоит из:

Пояснительной записки.

Пояснительная записка включает следующие разделы - введение, общая часть, анализ и проектирование, реализация программного модуля, тестирование программного модуля, заключение, список использованных источников, приложения.

Раздел «Введение» содержит актуальность работы, цели, задачи, объекты исследования и предмет исследования предметной области.

Раздел «Общая часть» содержит описание проектируемой предметной области, обоснованной выбором темы, описанием бизнес –процессов, направленных на достижение поставленной цели. Перечень документов, лиц, их задачи

Раздел «Анализ и проектирование содержит информацию о проектировании функциональной архитектуры программного средства, включающую в себя описание функциональных задач, решаемых с помощью проектируемой системы или модуля с

помощью диаграмм: диаграмма IDEF0, 0 и 1 уровней. Описание типа архитектуры, расположения уровня бизнес-логики, UML-диаграммы классов, диаграмма вариантов использования, диаграмма последовательности. Проектирование базы данных с помощью ER-диаграммы и формирование спецификации требований программного средства, указывая системные характеристики, пользовательские требования, бизнес-правила, функциональные требования.

Раздел «Реализация программного модуля» содержит информацию о разработке объектов базы данных в выбранной системе управления базой данных, включающую в себя писание объектов: таблиц, хранимых процедур, триггеров, представлений, функций, а также реализация функциональной архитектуры программного средства с разработкой форм, фрагментов программного кода и запросов.

Раздел «Тестирование программного модуля» содержит информацию о разработанных тест-кейсах с использованием специального инструментального средства для разработанного программного модуля с описанием и представленными тест-кейсами.

Список используемых источников состоит из 6 книг, 4 статей и 4 официальных документах по технологиям.

Заключение содержит итоги проведенной работы.

Пояснительная записка содержит следующие приложения – ссылка на систему контроля версий.

Пояснительная записка состоит из 44 стр, 21 рисунков, 13 таблиц.

1 ОБЩАЯ ЧАСТЬ

1.1 Описание проектируемой предметной области

Выбор темы курсовой работы обусловлен необходимостью автоматизации одного из ключевых, но часто недооценённых бизнес-процессов в современной организации — учёта и обработки заявок на устранение неисправностей техники. В ходе производственной практики по профессиональному модулю ПМ.01 «Разработка модулей программного обеспечения для компьютерных систем» было выявлено, что в организации данный процесс осуществляется не системно: используются электронные таблицы, устные обращения через телефонию. Это приводит к потере заявок, дублированию обращений, отсутствию контроля сроков и снижению общей эффективности технической поддержки.

Разработка программного модуля, специализированного на управлении заявками на неисправности техники, позволяет не только устранить указанные недостатки, но и заложить основу для последующей интеграции в более масштабные системы управления ИТ-инфраструктурой. Таким образом, тема работы является практически значимой, соответствует требованиям современной инженерии программного обеспечения и отражает актуальные задачи цифровой трансформации внутренних процессов.

Бизнес-процессы по проектируемой предметной области:

Регистрация заявки:

Заказчик (сотрудник организации, который может как быть техником, так и не быть им) сообщает о неисправности техники, указывая: уникальный инвентарный номер оборудования, описание проблемы, свои ФИО, комментарий что не так с работой техники. Система автоматически фиксирует дату подачи заявки.

Приём и первичная обработка заявки:

Сотрудник технического отдела принимает заявку, проверяет корректность данных и назначает исполнителя.

Исполнение заявки:

Назначенный исполнитель выполняет диагностику и ремонт оборудования. По завершении он указывает статус выполнения, дату завершения и, если это необходимо, комментарий.

Документы:

Заявка на ремонт техники - основной документ, фиксирующий факт обращения. Содержит:

– ФИО заказчика;

– Инвентарный номер;
– Описание неисправности;
– Дата регистрации;
– Назначенный исполнитель;
– Комментарий исполнителя;
– Дата подачи заявки и дата выполнения.

Лица и роли, участвующие в экосистеме программного средства можно посмотреть в соответствии с таблицей 1.

Таблица 1 - Перечень участников бизнес-процесса, их роли и задачи

Участник	**Роль**	**Основные задачи**
Заказчик	Инициатор заявки	Подача заявки на ремонт, указание характеристик неисправности и данных об оборудовании.
Сотрудник технического отдела	Диспетчер	Приём, проверка и первичная обработка заявок, назначение исполнителя.
Исполнитель	Ответственн ый за ремонт	Диагностика, устранение неисправности, фиксация результата в системе
Руководитель технического отдела	Аналитик	Мониторинг исполнения заявок

1.2 Анализ существующих решений

Первым аналогом программного средства является GLPI. GLPI расшифровывается как Gestionnaire Libre de Parc Informatique (Свободный менеджер ИТ-парка) и представляет собой бесплатный программный пакет для управления активами и ИТ-инфраструктурой, который предоставляет функции службы поддержки ITIL, отслеживание лицензий и аудит программного обеспечения.

Основной функционал:

– Обработка запросов;
– Управление инцидентами и проблемами;
– Система управления активами и конфигурациями;
– Управление изменениями;
– Управление знаниями;
– Управление контрактами;
– Управление финансами ИТ-услуг;
– Управление инфраструктурой центров обработки данных;

– Управление программным обеспечением и лицензиями;

– Разделение организаций;

– Управление проектами;

– Планирование мероприятий;

Посмотреть панель ресурсов можно в соответствии с рисунком 1.1.

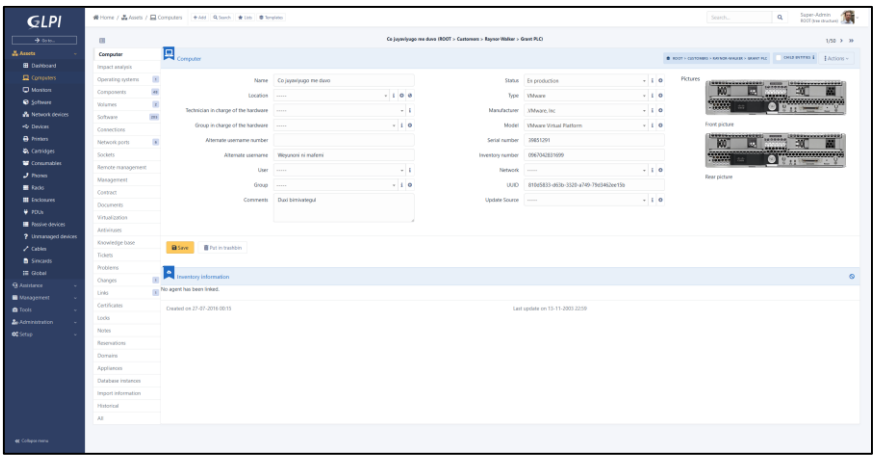


Рисунок 1.1 - Панель ресурсов

Посмотреть панель мониторинга можно в соответствии с рисунком 1.2.

Рисунок 1.2 - Панель мониторинга

Вторым аналогом программного средства является Freshservice. Freshservice — это облачная платформа управления ИТ-услугами ITSM, разработанная компанией Freshworks. Решение построено в соответствии с лучшими практиками ITIL и ориентировано на автоматизацию и упрощение процессов ИТ-поддержки, управления активами и обслуживания пользователей в организациях любого масштаба, являясь коммерческой платформой.

Ценовые категории:

– 1 085 руб. мес. - Для небольших команд, которые только начинают внедрять свою первую службу поддержки и отказываются от общих почтовых ящиков;
– 4 655 руб. мес. Для ИТ-команд, формирующих базовые практики, чтобы перейти от реактивной поддержки к упорядоченной и эффективной доставке услуг;
– 9 405 руб. мес. - Для зрелых команд, разрушающих информационные «острова» и объединяющих управление сервисами между разными подразделениями.

Осуществляемый функционал:

– Обработка запросов через многофункциональный сервис-деск;
– Управление инцидентами, проблемами и изменениями по ITIL-методологии;
– Полноценная система управления ИТ-активами (ITAM) с автоматическим обнаружением оборудования и ПО;
– Каталог услуг для конечных пользователей с возможностью самообслуживания;
– Гибкие SLA-политики с контролем сроков исполнения заявок;
– База знаний Knowledge Base для быстрого поиска решений и снижения нагрузки на поддержку;
– Управление заявками по категориям: ИТ, HR, административные вопросы и др.;
– Автоматизация рутинных задач с помощью правил и workflow-сценариев;
– Интеграция с популярными инструментами Microsoft Teams, Slack, Google Workspace, Zoom, Active Directory;
– Мобильное приложение для агентов и пользователей;
– Аналитика и отчёты в реальном времени по ключевым метрикам поддержки;
– Поддержка нескольких организаций Multi-Tenant в рамках одного аккаунта.

Посмотреть окно управления можно в соответствии с рисунком 1.3.

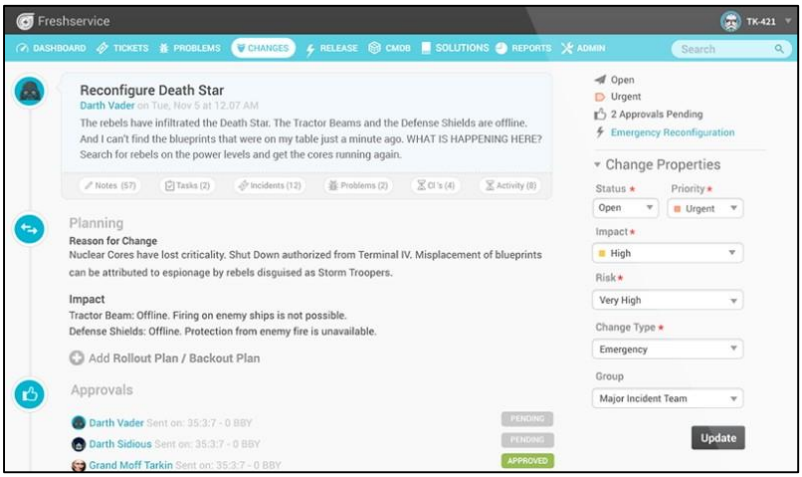


Рисунок 1.3 - Окно управления

LiveSklad — это российская облачная система управления сервисными центрами и мастерскими, ориентированная на учёт заявок на ремонт техники, управление сотрудниками и контроль выполнения работ. Система разработана для компаний, оказывающих техническое обслуживание оборудования, и позволяет эффективно организовывать внутренние и внешние сервисные процессы. Данный продукт подходит лишь частично к проектируемой области.

Система предлагается по подписке с тарифами от 900 руб. мес. для 2 сотрудников и 1 мастерской до 14 500 руб. мес. до 80 сотрудников и 20 мастерских, а также доступен бесплатный пробный период на 7 дней.

Основной функционал:

- Приём и регистрация заявок на ремонт техники;
- Учёт оборудования с привязкой к инвентарным номерам и типам устройств;
- Назначение исполнителей (мастеров, инженеров) и распределение задач по мастерским;
- Контроль сроков выполнения заявок и отслеживание статусов («в работе», «ожидает запчасти», «выполнено»);
- Поддержка составных заявок с несколькими устройствами и неисправностями;
- Управление штатом сотрудников и распределение ролей;
- Организация работы нескольких мастерских (подразделений) в единой системе;
- Ведение истории обращений по каждому устройству или клиенту;
- Гибкая тарифная модель с учётом числа сотрудников и мастерских;
- Возможность адаптации под внутренние процессы организации в том числе для ИТ-поддержки сотрудников.

Посмотреть окно управления можно в соответствии с рисунком 1.4.

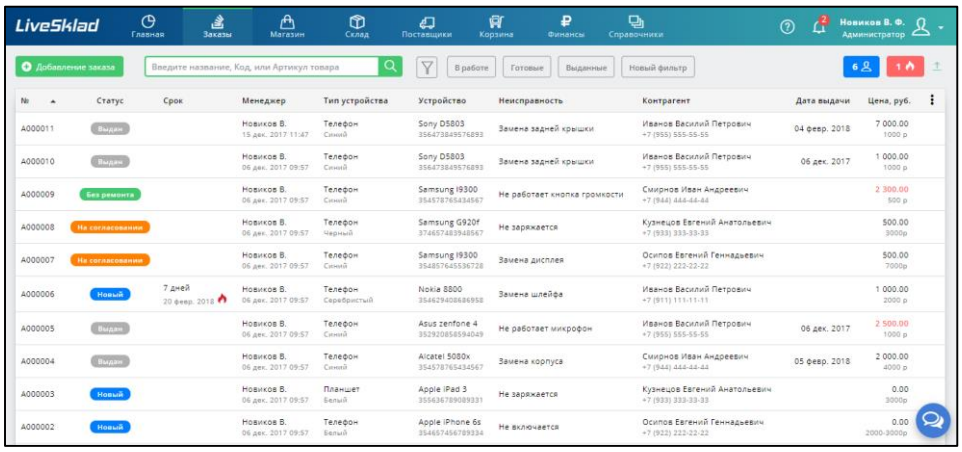


Рисунок 1.4 - Окно заказов

2 АНАЛИЗ И ПРОЕКТИРОВАНИЕ

2.1 Проектирование функциональной архитектуры программного средства

IDEF0 диаграмма нулевого уровня по приложению. В результате процесса «составить заявку» необходимы входные данные, такие как: «данные сотрудника», «данные техника», «оборудование», «описание проблемы» для того, чтобы получить на выходе «заполненная форма на печать» под управлением регламентирующих документов – «должностная инструкция», «техническая документация», «согласие на обработку персональных данных», «руководство пользователя», «ГОСТ Р7.0.97-2016».

Механизмами, которые взаимодействуют с процессами, являются: «сотрудник», «техник», «программа», «интерфейс», IDEF0 диаграмма нулевого уровня представлена в соответствии с рисунком 2.1.

Рисунок 2.1 — IDEF0 нулевого уровня

IDEF0 первого уровня это декомпозиция IDEF0 нулевого уровня предназначенная для моделирования данных и информации в системах.

В результате декомпозиции IDEF0 диаграммы 0 уровня происходит декомпозиция блока «составить заявку» на 4 подпроцесса: «заполнение формы», «отправить заявку техникам», «обработать заявку» и «подтвердить заявку и сохранить». В результате распределения входных данных по под процессам. Получили следующее:

Входные данные: «данные сотрудника», «описание проблемы», «оборудование» входят в подпроцесс «заполнение формы»;

Входные данные «заполненная форма» входит в подпроцесс «отправить заявку техникам»;

Входные данные «заявка с данными сотрудника», «оборудование», «данные техника», входят в под процесс «обработать заявку»;

Входные данные «заполненная заявка» входит в подпроцесс «подтверждать заявку и сохранять ее».

Управляющая документация «техническая документация» и «согласие на обработку персональных данных» входят в под процессы «заполнение формы», «обработать заявку»;

Управляющая документация «ГОСТ Р7.0.97-2016» входит в под процессы «заполнение формы», «отправить заявку техникам», «обработать заявку»;

Управляющая документация «руководство пользователя» входит в под процессы «заполнение формы», «отправить заявку техникам», «обработать заявку», «отправить документ на печать»;

Управляющая документация «должностная инструкция» входит в подпроцесс «заполнение формы».

Механизмы «техники», «интерфейс», «программа» принимают участие в под процессах «заполнение формы», «отправить заявку техникам», «обработать заявку», «отправить документ на печать»;

Механизмы «сотрудник» входит в под процессы «заполнение формы» и «отправить заявку техникам».

Благодаря этим входным данным, управляющей документации и механизмам, получены на выходе «подтверждать заявку и сохранять ее» в соответствии с рисунком 2.2.

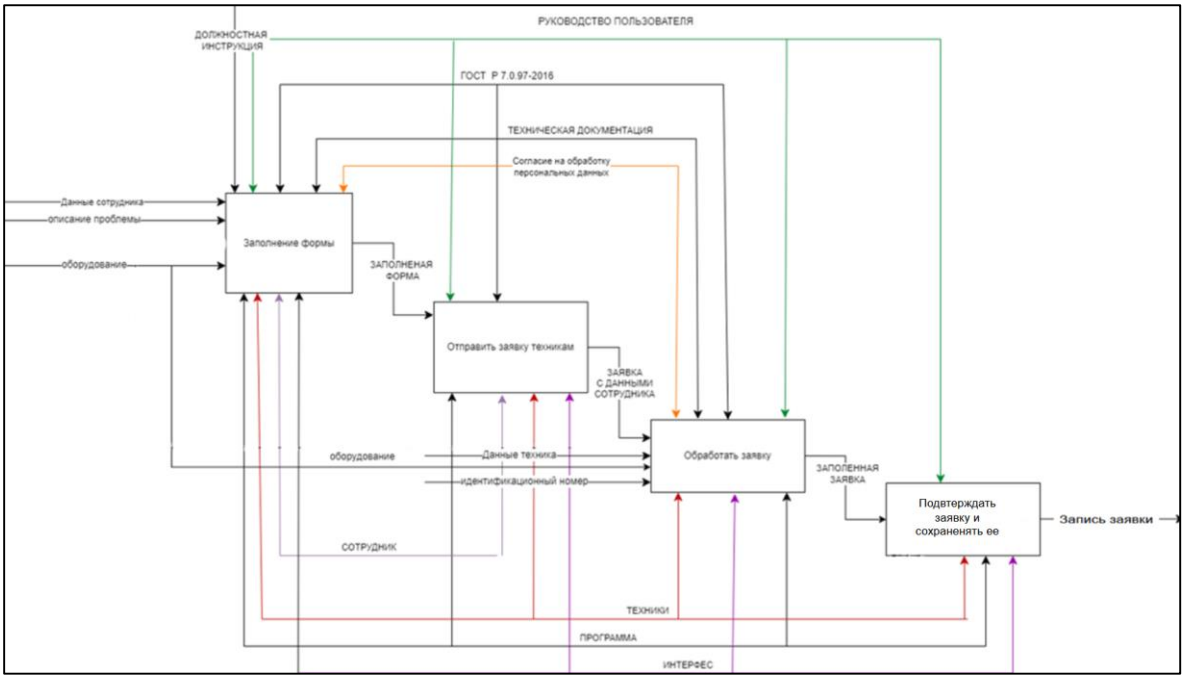


Рисунок 2.2 — IDEF0 нулевого уровня

Система построена на типе архитектуры – многоуровневая клиент-серверная. Посмотреть архитектуру можно в соответствии с таблицей 2.

Таблица 2 – Архитектура приложения

Уровень	Назначение	Технология
Уровень представления	Веб-интерфейс для взаимодействия с пользователями.	веб-браузер
Уровень бизнес-логики	Обработка бизнес-правил, валидация данных, управление workflow заявок	Django
Уровень данных	Хранение и управление данными.	PostgreSQL СУБД

Конкретная реализация:

– Бизнес-логика размещена на сервере приложения;
– Клиентская часть отвечает только за отображение данных и ввод информации;
– Все основные процессы обрабатываются на сервере;
– СУБД PostgreSQL обеспечивает хранение и целостность данных.

Была разработана диаграмма классов. Класс «ТипОборудования» включает в себя открытые для всех поля класса с такими атрибутами как: «наименование» типа string, «id» типа int. Имеет метод: «получитьТипы()». «ТипОборудования» может быть у одного «оборудования» или нескольких одновременно.

Класс «Кабинет» включает в себя открытые для всех поля класса с такими атрибутами как: «id» типа int. Имеет метод: «получитьКабинеты()». «Кабинет» может быть закреплён за

одним «сотрудником» или несколькими одновременно. В кабинете может размещаться одно «оборудование», несколько или вообще не быть оборудования.

Класс «Отдел» включает в себя открытые для всех поля класса с такими атрибутами как: «наименование» типа string, «id» типа int. Имеет метод: «получитьОтделы()». «Отдел» может включать в себя одного «сотрудника» или нескольких одновременно, при этом сотрудники обязательно привязаны к определённому отделу.

Класс «Должность» включает в себя открытые для всех поля класса с такими атрибутами как: «наименование» типа string, «id» типа int. Имеет метод: «получитьДолжности()». «Должность» может быть у одного «сотрудника» или нескольких одновременно, при этом она обязательно есть у каждого сотрудника.

Класс «Сотрудник» включает в себя открытые для всех поля класса с такими атрибутами как: «инн» типа string, «фамилия» типа string, «имя» типа string, «отчество» типа string, «номерТелефона» типа string. Имеет методы: «авторизоваться()», «получитьРоль()», «ПросмотретьСвоиЗаявки()», «ЗаполнитьФорму()», «ОтправитьФорму()». «Сотрудник» может соответствовать только одной «должности», одному «отделу» и одному «кабинету», при этом обязательно соответствует. Сотрудник может быть, как заказчиком, так и исполнителем в одной или нескольких «заявках».

Класс «Оборудование» включает в себя открытые для всех поля класса с такими атрибутами как: «инвентарныйНомер» типа string, «комплектация» типа string. Имеет методы: «получитьИнформацию()», «изменитьСтатус()». «Оборудованию» соответствует только один «кабинет» и только один «тип», при этом обязательно соответствует. Оборудование может быть, как в одной «заявке», так и в нескольких. «СтатусПочинки» определяет состояние оборудования.

Класс «Заявка» включает в себя открытые для всех поля класса с такими атрибутами как: «номерАкта» типа string, «описаниеНеисправности» типа string, «датаРегистрации» типа DateTime, «датаВыполнения» типа DateTime, «комментарийИсполнителя» типа string. Имеет методы: «создатьЗаявку()», «назначитьИсполнителя()», «изменитьСтатус()», «получитьЗаявкиПоДате()», «ОтредактироватьЗаявку()», «СохранитьЗаявку()». «Заявке» соответствует только один «сотрудник», только один «исполнитель» и только одно «оборудование». «СтатусЗаявки» определяет текущее состояние заявки.

Класс «СтатусЗаявки» является перечислением и включает в себя открытые для всех поля класса с такими атрибутами как: «Новая» типа int = 1, «В_работе» типа int = 2, «Выполнена» типа int = 3, «Отменена» типа int = 4. «СтатусЗаявки» может быть у одной

«заявки» или нескольких одновременно. Посмотреть диаграмму классов можно в соответствии с рисунком 2.3.

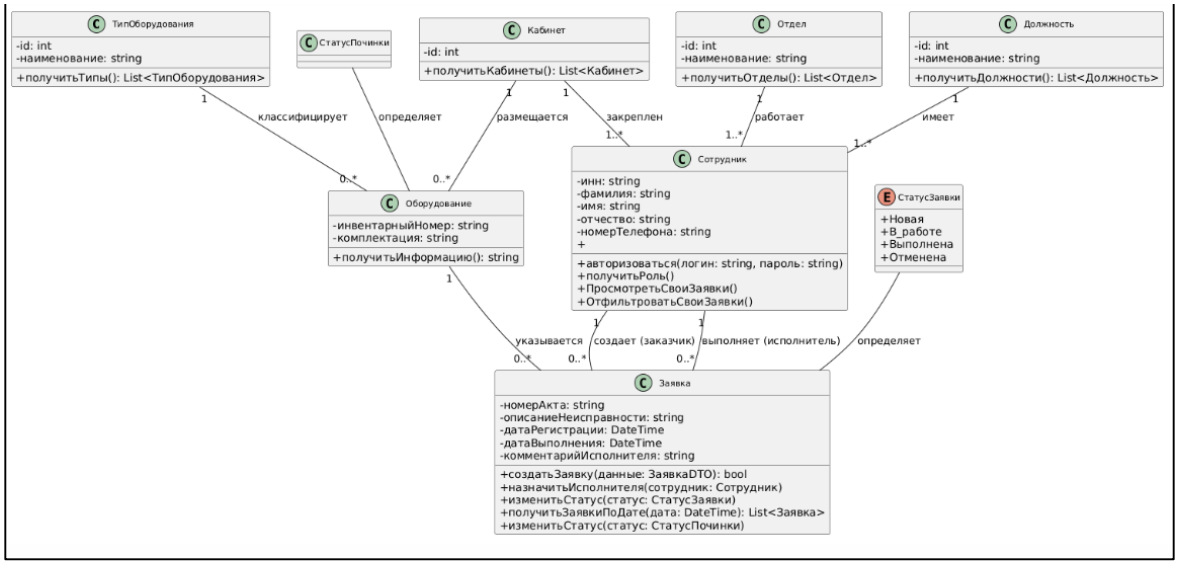


Рисунок 2.3 — Диаграмма классов

На диаграмме вариантов использования можно увидеть двух экторов: «сотрудник» и «техник». У каждого эктора есть свои прецеденты, которые отвечают за свою функцию. Что бы увидеть каким прецедентам какие экторы соответствуют, можно обратиться к таблице прецедентов в соответствии с таблицей 3.

Таблица 3 — Таблица прецедентов

Эктор	Прецедент
Сотрудник	Открыть форму для шаблона
Сотрудник	Заполнить данные
Сотрудник	Отправить заполненную форму
Сотрудник	Закрыть форму
Сотрудник	Просматривать свои заявки
Техник	Просматривать все заявки
Техник	Изменять заявку
Техник	Фильтровать заявки

Диаграмма вариантов использования показывает взаимодействие экторов и их прецедентов. Эктор «техник» может «просматривать все заявки», «изменять заявку» и «фильтровать заявки». Так же эктор «техник» обобщается в эктора «сотрудник».

Эктор «сотрудник» может «просматривать свои заявки» и «открывать форму для шаблона». Открыв форму для шаблона «сотрудник» может «закрыть форму», а может и не закрывать. Эктор может «заполнить данные» а может не заполнять, но если заполнит, то обязательно «отправить заполненную форму». Посмотреть диаграмму можно в соответствии с рисунком 2.4.

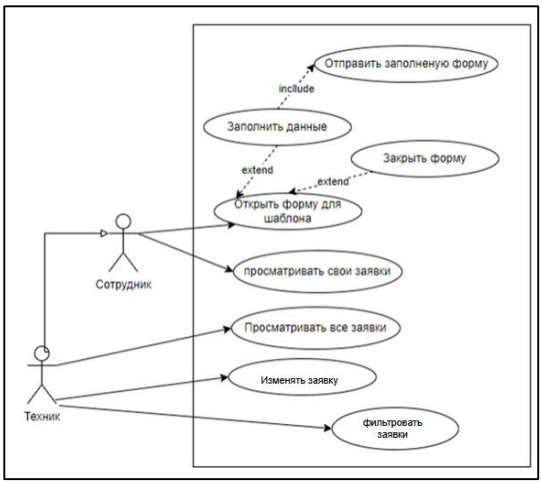


Рисунок 2.4 — Диаграмма вариантов использования

Диаграмма последовательности с основными и уникальными действиями включает себя таких экторов как: «Пользователь», «Интерфейс», «Система».

В диаграмме, когда все действия идеальны и система не перестает работать, рассматривается процесс от лица пользователя под должностью «техник». В основной работе участвуют три эктора: «Пользователь», «Интерфейс», «Система». «Пользователь» подает запросы системе через нажатия на кнопки интерфейса. «Интерфейс» в ответ на действия пользователя передает запрос эктору «Система». «Система» обрабатывает запрос, вносит изменения системы и дает ответ на запрос эктора «Интерфейс», после чего «Интерфейс» открывает окно, которое запросил «Пользователь». Диаграмма классов представлена в соответствии с рисунком 2.5.

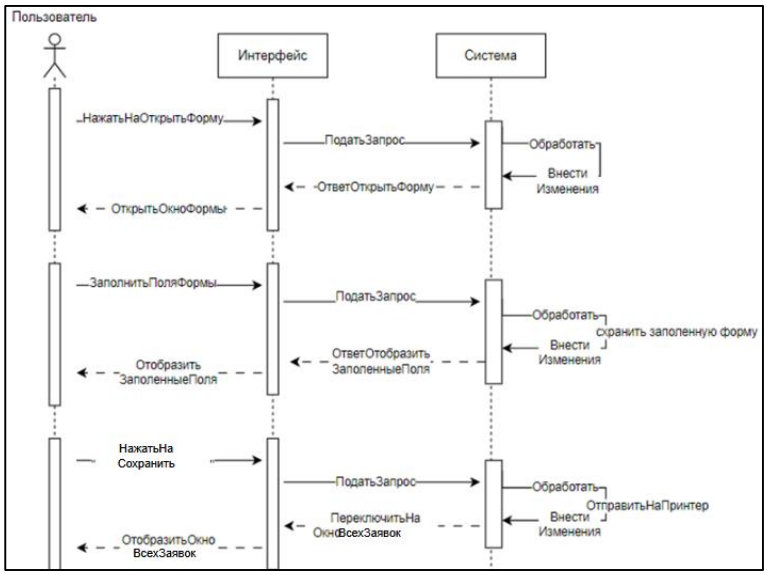


Рисунок 2.5 — Диаграмма последовательности

2.2 Проектирование структуры базы данных

В ходе проектирования структуры базы данных была собрана ER-диаграмма для реализации учета заявок неисправностей с оборудованием в организации. Посмотреть детально ER-диаграмму можно в соответствии с рисунком 2.6.

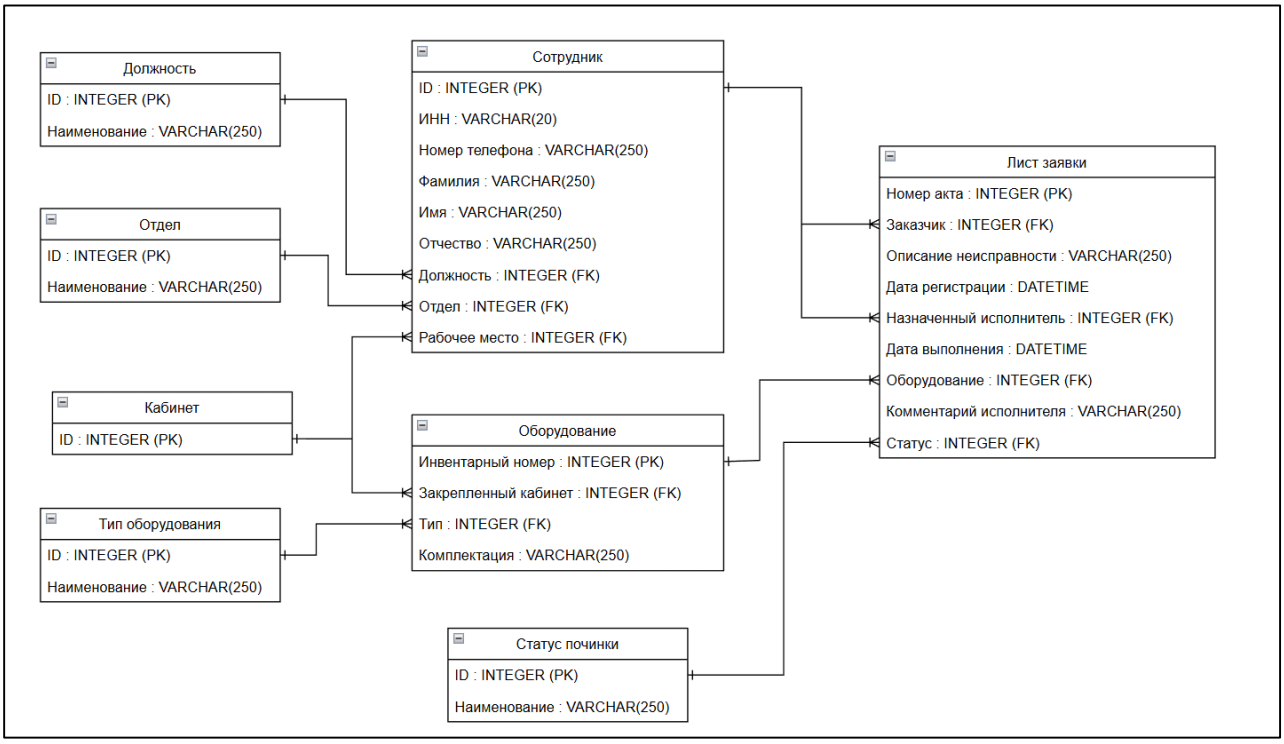


Рисунок 2.6 — ER-диаграмма

ER-диаграмма состоит из 8 таблиц:

Должность включает в себя «ID» и «Наименование должности». «ID» — первичный ключ. Должность может быть, как у одного, так и у нескольких сотрудников при этом, она обязательно есть.

Отдел включает в себя «ID» и «Наименование отдела». «ID» — первичный ключ. Отдел может включать в себя как одного, так и нескольких сотрудников, при этом обязательно привязаны к определенному отделу.

Кабинет состоит только из первичного ключа. Кабинет может быть выделен как одному, так и нескольким сотрудникам, и, обязательно выделен. В кабинете может быть одно оборудование, несколько или вообще не быть оборудования.

Сотрудник состоит из следующих полей: «ID» - первичный ключ, «ИНН», «Номер телефона», «Фамилия», «Имя», «Отчество», «Должность» - внешний ключ, «Отдел» - внешний ключ, «Рабочее место» - внешний ключ. Сотруднику соответствуют только одна

должность, один отдел и один кабинет и обязательно соответствует. Сотрудник может быть, как заказчиком, так и техником в одном лице и быть в одном или нескольких листах заявки, при этом обязательно быть в листе заявки.

Тип оборудования состоит из «ID» и «Наименования типа». «ID» - первичный ключ, «Наименование типа». Тип оборудования может быть, как у одного, так и у нескольких оборудований и обязательно есть.

Оборудование состоит из «Инвентарного номера» - первичный ключ, «Закрепленный кабинет» - внешний ключ, «Тип» - внешний ключ, «Комплектация», «Статус починки» - внешний ключ. Оборудованию соответствует только один кабинет и только один тип, при этом, обязательно соответствует. Оборудование может быть, как в одном листе заявки, так и в нескольких.

Статус починки состоит из «ID» и «Наименования статуса». «ID» - первичный ключ, «Наименование статуса». Статус починки может быть, как у одной заявки, так и у нескольких и обязательно есть.

Лист заявки состоит из «Номер акта» - первичный ключ, «Заказчик» - внешний ключ, «Описание неисправности», «Дата регистрации», «Назначенный исполнитель» - внешний ключ, «Дата выполнения», «Оборудование» - внешний ключ, «Комментарий исполнителя», «Статус починки». Листу заявки соответствует только один сотрудник, только один исполнитель, только один статус и только одно оборудование и обязательно соответствует.

2.3 Формирование спецификации требований программного средства

Были составлены пользовательские требования к заказчику и технику:

- Авторизация в системе;
- Создание заявки на ремонт с указанием инвентарного номера, описания неисправности, ФИО и комментария;
- Просмотр статуса своих заявок;
- Просмотр всех поступивших заявок;
- Проверка корректности данных заявки;
- Изменение статуса;
- Указание даты и времени выполнения;
- Закрытие заявки;
- Сохранение заявки;

Функциональные требования:

- Авторизация пользователей;

– Создание заявки на ремонт;
– Автофиксация даты регистрации заявки;
– Назначение исполнителя на заявку;
– Добавление комментариев к заявке;
– Поиск и фильтрация заявок;

Были составлены бизнес-правила в соответствии с таблицей 4.

Таблица 4 - Бизнес правила

Правило	Описание
Обязательные поля заявки	Заявка не создаётся без: ФИО заказчика, инвентарного номера, описания неисправности
Уникальность инвентарного номера	Инвентарный номер оборудования уникален в системе
Назначение исполнителя	На одну заявку — только один исполнитель
Срок выполнения	Заявка выполняется в течение нужного количества рабочих дней с регистрации в зависимости от типа поломки и типа оборудования
Доступ к данным	Пользователь видит только заявки, связанные с ним в отношении заказчика
Редактирование заявки	Редактирование возможно только в статусе «Новая» для заказчика
Комментарий при закрытии	При закрытии заявки исполнитель добавляет комментарий о работе
Привязка оборудования	Оборудование привязано к кабинету и типу обязательно

Детальные спецификации по каждым таблицам главных объектов представлены на таблицах 5-7.

Таблица 5 - Спецификация объекта «Заявка»

Поле	Тип	Обязательно	Описание
НомерАкта	INTEGER	Да	Первичный ключ
Заказчик	INTEGER (FK)	Да	Ссылка на сотрудника
ОписаниеНесправности	VARCHAR(250)	Да	Описание проблемы
ДатаРегистрации	DATETIME	Да	Автозаполнение при создании
НазначенныйИсполнитель	INTEGER (FK)	Да	Ссылка на сотрудника
ДатаВыполнения	DATETIME	Да	Заполнение при закрытии
Оборудование	INTEGER (FK)	Да	Ссылка на оборудование
КомменнтарийИсполнителя	VARCHAR(250)	Нет	Заполнение исполнителем
Статус	VARCHAR(250)	Да	Новая/В работе/Выполнена/Закрыта

Таблица 6 - Спецификация объекта «Сотрудник»

Поле	Тип	Обязательно	Описание
ID	VARCHAR(250)	Да	Первичный ключ
ИНН	VARCHAR(250)	Да	ИНН
Фамилия	VARCHAR(250)	Да	Фамилия сотрудника
Имя	VARCHAR(250)	Да	Имя сотрудника
Отчество	VARCHAR(250)	Да	Отчество сотрудника
НомерТелефона	VARCHAR(250)	Да	Контактный телефон
Должность	INTEGER (FK)	Да	Ссылка на должность
Отдел	INTEGER (FK)	Да	Ссылка на отдел
Кабинет	INTEGER (FK)	Да	Ссылка на кабинет

Таблица 7 - Спецификация объекта «Оборудование»

Поле	Тип	Обязательно	Описание
Инвентарный номер	INTEGER (FK)	Да	Первичный ключ
ЗакрепленныйКабинет	INTEGER (FK)	Да	Ссылка на кабинет
Тип	INTEGER (FK)	Да	Ссылка на тип оборудования
Комплектация	VARCHAR(250)	Да	Описание комплекции

Были указаны также и системные характеристики. Они описывают технические требования к программе и нужны для обоснования выбора технологий, обеспечения совместимости компонентов и планирования ресурсов. Фиксация версий ПО, СУБД и протоколов гарантирует стабильность работы, безопасность данных и упрощает дальнейшее сопровождение системы. Системные характеристики представлены в таблице 8.

Таблица 8 – Системные характеристики

Характеристика	Значение
Тип системы	Веб-приложение для учёта заявок на ремонт техники
Платформа	Кроссплатформенное
СУБД	PostgreSQL 14+
Язык разработки	Python 3.10+
Фреймворк	Django 4.x
Архитектура	Клиент-серверная, 3 уровня
Протокол связи	HTTPS
Поддерживаемые браузеры	Chrome, Firefox, Edge
Объем бд	До 10 Гб

3 РАЗРАБОТКА ПРОГРАММНОГО МОДУЛЯ

3.1 Разработка объектов базы данных в выбранной СУБД

Для разработки базы данных и хранения данных была выбрана СУБД — postgresql. Данный продукт является мощной объектно-реляционной СУБД с открытым исходным кодом. Она поддерживает сложные SQL запросы и типы данных в виде массивов, что хорошо сочетается с веб-приложениями.

База данных состоит из 8 таблиц со связями "Один-ко-многим", в них не предусмотрена сложная связь «Многие-ко-многим», что связано и так с логичной цепочкой последовательности. Сложная логика, как триггеры или процедуры не предусмотрена в моделях. Весь фактический функционал реализован в функциональной архитектуре приложения. Просмотреть модели базы данных с описанием можно в соответствии с листингом 1.

Листинг 1 –Модели базы данных

```
from django.db import models
from django.contrib.auth.models import User
#-----------------------------------------------------------------
#ДОЛЖНОСТЬ по совместительству Роли пользователей
class Role(models.Model):
    ROLE_CHOICES = [
        ('technician', 'Техник'), #по совмесительству администратор
        ('employee', 'Сотрудник'), #просто сотрудник который может
создать заявку с определенными формами
    ]
    role_name = models.CharField(max_length=250,
choices=ROLE_CHOICES, unique=True)
#-----------------------------------------------------------------

#ОТДЕЛ
class Department(models.Model):
    id = models.IntegerField(primary_key=True, verbose_name="ID")
    name =
models.CharField(max_length=250,verbose_name="Наименование отдела")
#-----------------------------------------------------------------

#КАБИНЕТ
class Office(models.Model):
    id = models.IntegerField(primary_key=True, verbose_name="номер
кабинета")
#-----------------------------------------------------------------

#ТИП ОБОРУДОВАНИЯ
class EquipmentType(models.Model):
    id = models.IntegerField(primary_key=True, verbose_name="ID")
```

Продолжение листинга 1

```
    name =
models.CharField(max_length=250,verbose_name="Наименование типа")
#-----------------------------------------------------------------

#СТАТУС ЗАЯВКИ
class ApplicationStatus(models.Model):
    id = models.IntegerField(primary_key=True, verbose_name="ID")
    name =
models.CharField(max_length=250,verbose_name="Наименование  статуса")
#-----------------------------------------------------------------

#СОТРУДНИК
class Employee(models.Model):
id = models.CharField(max_length=20, primary_key=True,
verbose_name="ID")
    inn = models.CharField(max_length=20, verbose_name="ИНН")
#логин
number_phone= models.CharField(max_length=250,verbose_name="Номер
телефона")
    first_name =
models.CharField(max_length=250,verbose_name="Фамилия")
    name = models.CharField(max_length=250,verbose_name="Имя")
    second_name =
models.CharField(max_length=250,verbose_name="Отчество")
    position = models.ForeignKey(Role, on_delete=models.CASCADE,
verbose_name="Должность")
    department = models.ForeignKey(Department,
on_delete=models.CASCADE,verbose_name="Отдел")
    office = models.ForeignKey(Office, on_delete=models.CASCADE,
verbose_name="Рабочее место")
    password = models.CharField(max_length=250)
    last_login = models.DateTimeField(null=True, blank=True) #Время
последнего захода в приложениие
#-----------------------------------------------------------------

#ОБОРУДОВАНИЕ
class Equipment(models.Model):
    id = models.IntegerField(primary_key=True,
verbose_name="Инвентарный номер")
    set =
models.CharField(max_length=250,verbose_name="Комплектация")
    pin_office = models.ForeignKey(Office, on_delete=models.CASCADE,
verbose_name="Закрепленный кабинет")
    type = models.ForeignKey(EquipmentType,
on_delete=models.CASCADE,verbose_name="Тип оборудования")
#-----------------------------------------------------------------

#ЗАЯВКА
class RequestFix(models.Model):
    id = models.IntegerField(primary_key=True, verbose_name="Номер
акта")
```

Продолжение листинга 1

```
    description_problem =
models.CharField(max_length=250,verbose_name="Описание
неисправности")
    date_register = models.DateTimeField(null=True,
blank=True,auto_now_add=True, verbose_name="Дата регистрации")
    date_done = models.DateTimeField(null=True,
blank=True,verbose_name="Дата выполнения")
    comment_technician =
models.CharField(max_length=250,verbose_name="Комментарий
исполнителя")
    customer = models.ForeignKey(Employee, on_delete=models.CASCADE,
related_name="Сотрудник", verbose_name="Заказчик")
    given_technician = models.ForeignKey(Employee,null=True,
blank=True, on_delete=models.CASCADE, related_name="Техник",
verbose_name="Исполнитель")
    equipment = models.ForeignKey(Equipment,
on_delete=models.CASCADE, verbose_name="Оборудование")
    status_done_request = models.ForeignKey(ApplicationStatus,
on_delete=models.CASCADE, verbose_name="Статус заявки")
```

3.2 Разработка функциональной архитектуры программного средства

Вся функциональная архитектура прописана в файле view. Ознакомиться с работой функций его кодом можно далее.

Функция login_view обеспечивает аутентификацию пользователя в системе путем проверки введенных ИНН и пароля, при этом реализация выполнена через прямое сравнение пароля в открытом виде без хеширования, успешный вход фиксируется в сессии с сохранением роли и имени пользователя для последующего перенаправления на страницу техника или сотрудника в зависимости от должности. Реализация показана на рисунке 3.1.

Листинг 2 – Login_view

```
def login_view(request):
    if request.method == 'POST':
        inn = request.POST.get("username")
        password = request.POST.get("password")
        try:
            user = Employee.objects.get(inn=inn)
            if user.password == password:
                # Сохранение данных в сессии
                request.session['user_id'] = user.inn
                request.session['user_role']                        =
user.position.role_name
                request.session['user_name']  =  f'{user.first_name}
{user.name}'
                user.last_login = timezone.now()
```

Продолжение листинга 2

```
                user.save()

                # Перенаправление в зависимости от роли
                if user.position.role_name == 'Техник':
                    return redirect('show_technician')
                return redirect('show_customer')
            else:
                messages.error(request, 'Неверный пароль')
        except Employee.DoesNotExist:
            messages.error(request, 'Пользователь не найден')
    return render(request, 'login.html')
```

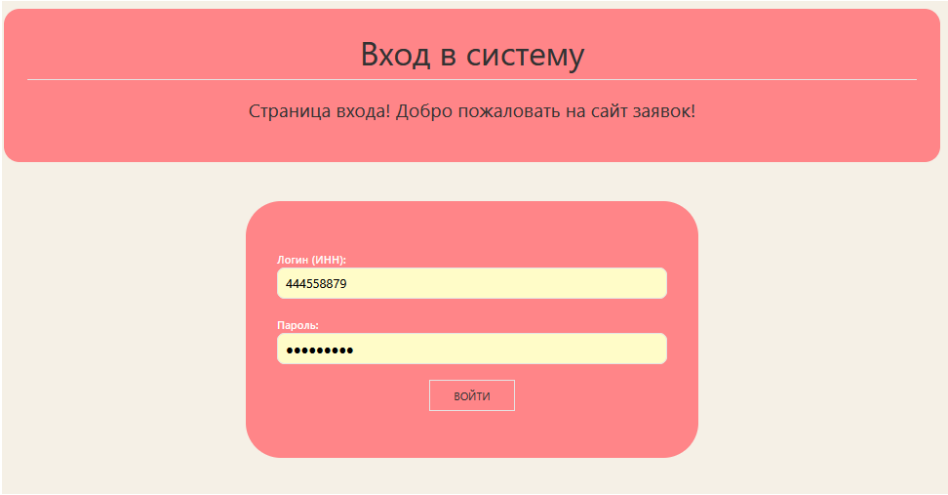


Рисунок 3.1 - Авторизация

Функция show_customer отображает личный кабинет сотрудника, формируя выборку заявок только для авторизованного пользователя, и реализует функционал поиска по тексту проблемы или оборудованию через Q-объекты, а также фильтрацию по статусам заявки. Реализация показана на рисунке 3.2.

Листинг 4 – Show_customer

```
def show_customer(request):
    # Проверка авторизации
    if 'user_id' not in request.session:
        return redirect('login_view')

    one_employee =
Employee.objects.get(inn=request.session['user_inn'])
    requests = RequestFix.objects.filter(customer=one_employee)

    # Поиск по проблеме или оборудованию
    search_query = request.GET.get('search', '')
    if search_query:
        requests = requests.filter(
            Q(description_problem__icontains=search_query) |
```

Продолжение листинга 4

```
            Q(equipment__set__icontains=search_query)
        )

    # Фильтрация по статусу
    status_filter = request.GET.get('status', '')
    if status_filter:
        requests =
requests.filter(status_done_request_id=status_filter)

    context = {'requests': requests, 'statuses':
ApplicationStatus.objects.all()}
    return render(request, 'show_customer.html', context)

    one_employee =
Employee.objects.get(inn=request.session['user_inn'])
    requests = RequestFix.objects.filter(customer=one_employee)
```

Рисунок 3.2 – Отображение заказчика с фильтрацией и поиском

Функция create_request_customer позволяет сотруднику создать новую заявку на ремонт, ограничивая выбор оборудования только теми единицами, которые закреплены за кабинетом сотрудника, и автоматически присваивает начальный статус и дату регистрации. Реализация показана на рисунке 3.3.

Листинг 5 – Create_request_customer

```
def create_request_customer(request):
    one_employee =
Employee.objects.get(inn=request.session['user_inn'])
    # Фильтрация оборудования по кабинету сотрудника
    equipment_list =
Equipment.objects.filter(pin_office=one_employee.office)
    default_status = ApplicationStatus.objects.get(id=1)
    if request.method == 'POST':
        try:
            req = RequestFix (
description_problem=request.POST.get('description_problem'),
                date_register=timezone.now(),
                customer=one_employee,
equipment=Equipment.objects.get(id=request.POST.get('equipment')),
                status_done_request=default_status)
            req.save()
            messages.success(request, 'Заявка создана!')
            return redirect('show_customer')
        except Exception as e:
            messages.error(request, f'Ошибка: {str(e)}')
return render(request, 'create_request_customer.html',
{'equipment_list': equipment_list})
```

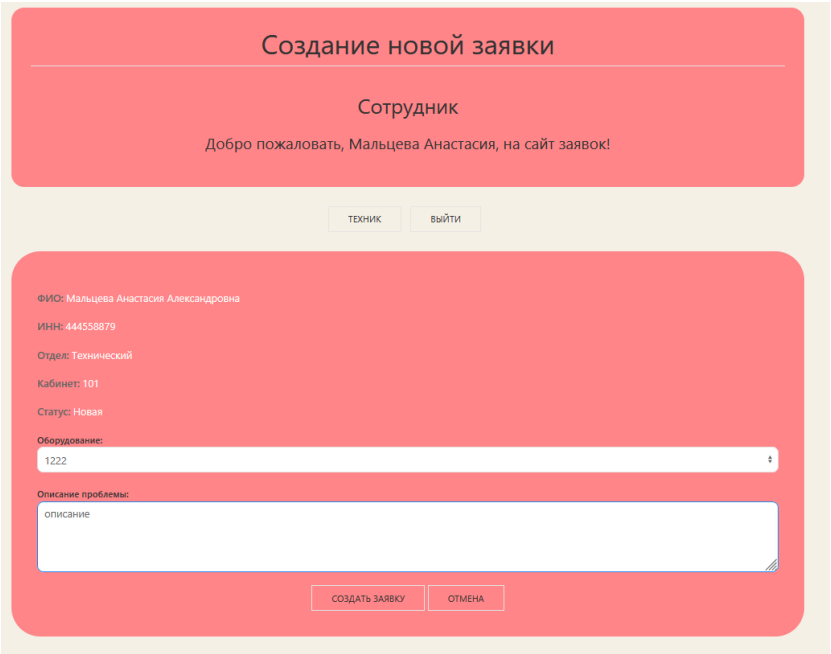


Рисунок 3.3 – Создание новой заявки от имени сотрудника

Функция show_technician предоставляет технику полный доступ ко всем заявкам системы с расширенными возможностями фильтрации по заказчику, оборудованию и комментариям, а также реализует сортировку списка по дате, статусу или имени заказчика для удобного управления нагрузкой. Реализация показана на рисунке 3.4.

Листинг 6 – Show_ technician

```
def show_technician(request):
    requests = RequestFix.objects.all()
    # Многосторонний поиск
    search_query = request.GET.get('search', '')
    if search_query:
        requests = requests.filter(
            Q(description_problem__icontains=search_query) |
            Q(customer__first_name__icontains=search_query) |
            Q(comment_technician__icontains=search_query)
        )
    # Сортировка результатов
    sort_by = request.GET.get('sort', 'id')
    if sort_by == 'date_register_desc':
        requests = requests.order_by('-date_register')
    elif sort_by == 'customer':
        requests = requests.order_by('customer__first_name')
    else:
        requests = requests.order_by('id')
    context = {'requests': requests, 'statuses':
ApplicationStatus.objects.all()}
    return render(request, 'show_technician.html', context)
```

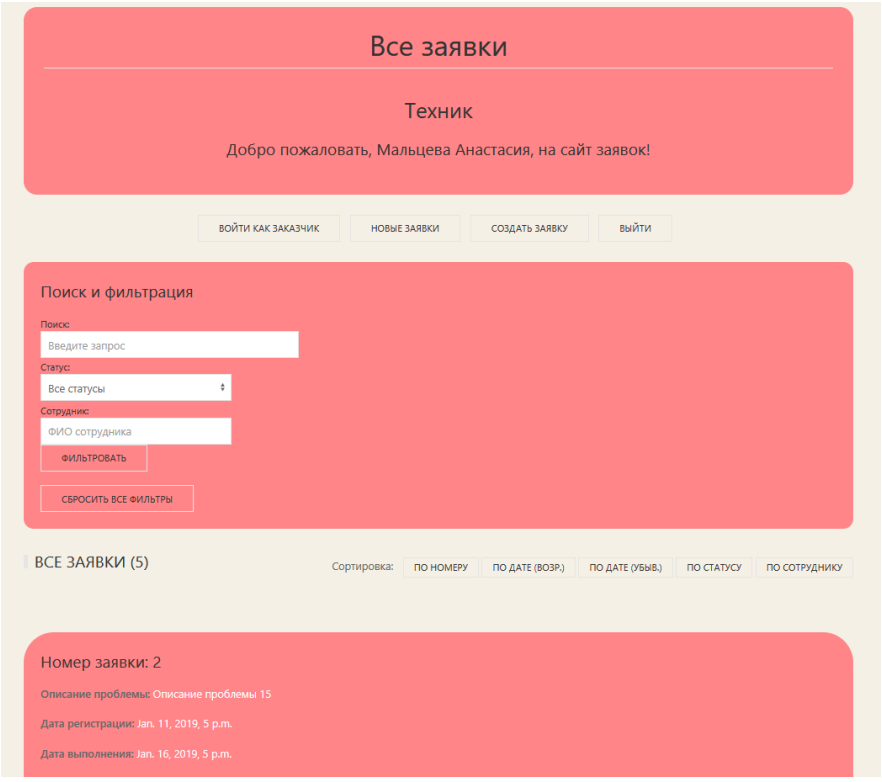


Рисунок 3.4 – Страница техника

Функция show_new_requests выбирает только новые заявки, используя сложную фильтрацию через Q-объекты для выявления записей без исполнителя, даты выполнения или с начальным статусом. Данная функция реализована для того, чтобы техник сразу мог просмотреть созданные и еще не взятые на починку. Реализация показана на рисунке 3.5.

Листинг 7 – Show_ new_requests

```
def show_new_requests(request):
    new_status = ApplicationStatus.objects.get(id=1)
    # Фильтр новых заявок через Q-объекты
    requests = RequestFix.objects.filter(
        Q(status_done_request=new_status) |
        Q(given_technician__isnull=True) |
        Q(date_done__isnull=True)
    )
    return render(request, 'show_new_requests.html', {'requests':
requests})
```

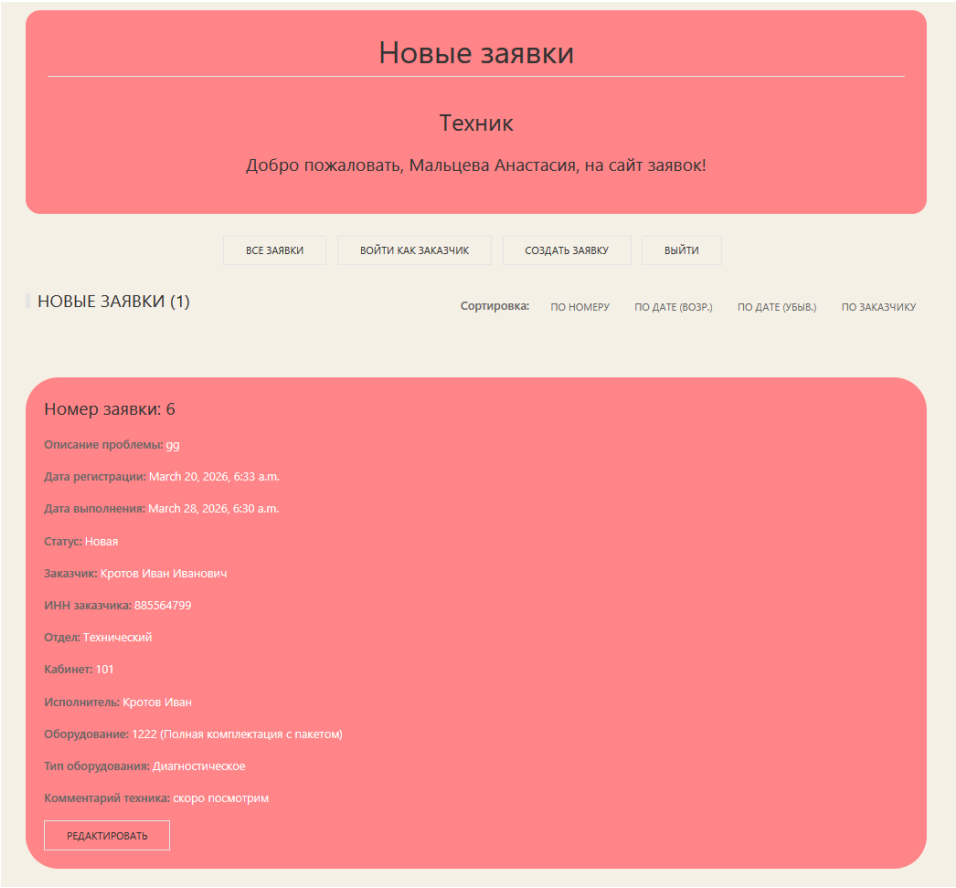


Рисунок 3.5 – Отображение страницы новых заявок

Функция create_request_technician позволяет техническому специалисту создавать заявки от имени любого сотрудника с полным контролем над всеми полями, включая выбор исполнителя и дат. Реализация показана на рисунке 3.6.

Листинг 8 – Create_request_technician

```
def create_request_technician(request):
    if request.method == 'POST':
        # Получение и валидация дат
        date_register = timezone.now()
        date_done = None
        # ... (парсинг дат из формы)
        customer =
Employee.objects.get(id=request.POST.get('customer'))
        equipment =
Equipment.objects.get(id=request.POST.get('equipment'))
        # Проверка соответствия кабинета
        if customer.office.id != equipment.pin_office.id:
            messages.error(request, 'Оборудование не из кабинета
заказчика!')

            return redirect('create_request_technician')
        req = RequestFix(
            customer=customer,
            equipment=equipment,
given_technician=Employee.objects.get(id=request.POST.get('given_te
chnician')),
status_done_request=ApplicationStatus.objects.get(id=request.POST.g
et('status'))
        )

        req.save()
        return redirect('show_technician')
    context = {
        'technicians':
Employee.objects.filter(position__role_name='Техник'),
        'customers': Employee.objects.all(),
        'equipment_list': Equipment.objects.all()
    }

    return render(request, 'create_request_technician.html',
context)
```

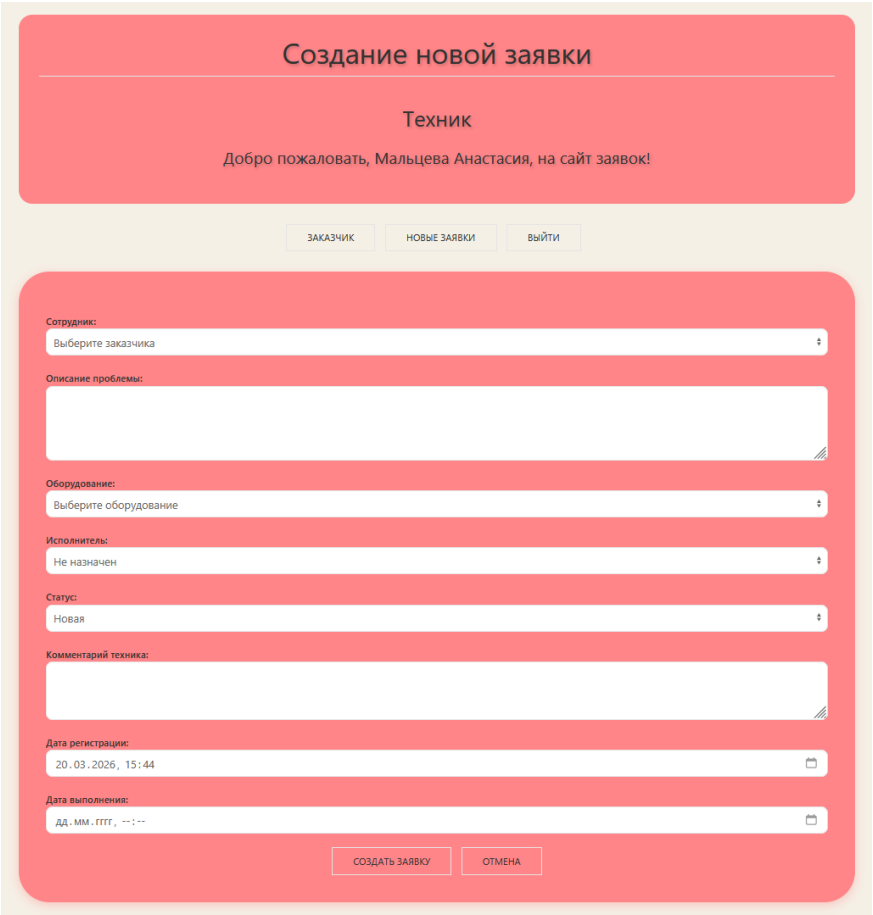


Рисунок 3.6 – Отображение создания заявки от роли техника

Функция edit_request реализует механизм редактирования существующей заявки техником, позволяя изменять исполнителя, статус, комментарий и даты выполнения. Реализация показана на рисунке 3.7.

Листинг 9 – Edit_request

```
def edit_request(request, request_id):
    req = get_object_or_404(RequestFix, id=request_id)

    if request.method == 'POST':
        req.description_problem =
request.POST.get('description_problem')
        req.comment_technician =
request.POST.get('comment_technician')
        req.status_done_request =
ApplicationStatus.objects.get(id=request.POST.get('status'))

        if request.POST.get('given_technician'):
            req.given_technician =
Employee.objects.get(id=request.POST.get('given_technician'))

        # Обновление дат
        if request.POST.get('date_done'):
```

Продолжение листинга 9

```
            req.date_done =
timezone.make_aware(datetime.strptime(request.POST.get('date_done'),
'%Y-%m-%dT%H:%M'))

        req.save()
        messages.success(request, 'Заявка обновлена!')
        return redirect('show_technician')

    return render(request, 'edit_request.html', {'request': req})
```

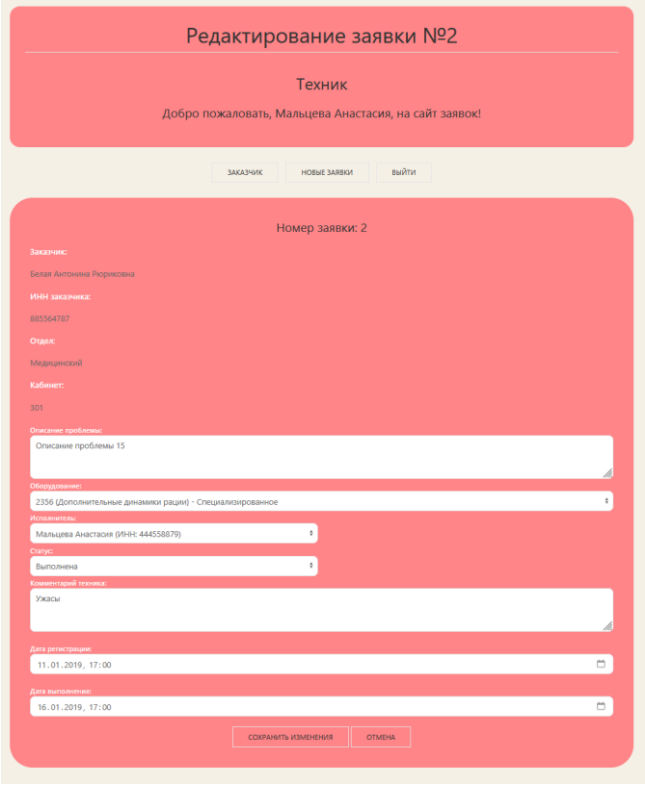


Рисунок 3.7 – Редактирование заявки

Функция delete_request обеспечивает удаление заявки из системы по идентификатору, доступное только для авторизованных техников.

Листинг 10 – Delete _request

```
def delete_request(request, request_id):
    if request.method == 'POST':
        try:
            req = RequestFix.objects.get(id=request_id)
            req.delete()
            messages.success(request, 'Заявка удалена')
        except RequestFix.DoesNotExist:
            messages.error(request, 'Заявка не найдена')
    return redirect('show_technician'
```

4 ТЕСТИРОВАНИЕ ПРОГРАММНОГО МОДУЛЯ

4.1 Разработка тест-кейсов

Этап тестирования является неотъемлемой частью жизненного цикла разработки программного обеспечения, позволяющей убедиться в соответствии реализованного функционала первоначальным требованиям и спецификации. В рамках данного раздела были разработаны тест-кейсы, описывающие последовательность действий пользователя и ожидаемые результаты работы системы для различных сценариев использования. Сценарии тестирования охватывают ключевые бизнес-процессы приложения, такие как авторизация пользователей, создание и обработка заявок, навигация между ролями заказчика и техника, а также фильтрация и сортировка данных. Каждый тест-кейс структурирован и содержит предусловия, пошаговое описание действий, ожидаемое поведение системы и фиксацию статуса прохождения, что обеспечивает системный подход к проверке работоспособности приложения. Посмотреть тест-кейсы можно в таблицах 9-13.

Таблица 9 - Тест-кейс №1

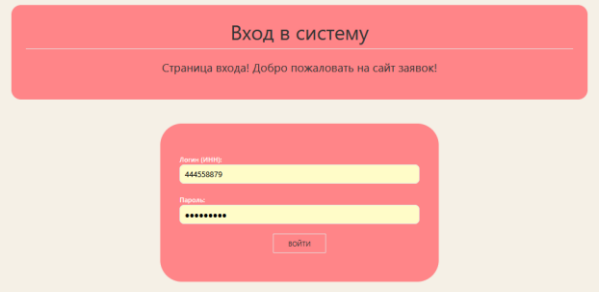
Номер	1
Название	Тестирование входа в аккаунт с ролью “Техник ”
Предусловие	--
Шаг №1	Запустить программу через python3 manage.py runserver и перейти на на страницу http://127.0.0.1:8000/ [student-pro@b206-1 projectBOM]$ python3 manage.py runserver Watching for file changes with StatReloader Performing system checks... System check identified no issues (0 silenced). February 24, 2026 - 05:09:17 Django version 4.2.27, using settings 'projectBOM.settings' Starting development server at http://127.0.0.1:8000/ Quit the server with CONTROL-C.
Результат шага №1	Отображение окна с наименованием «Вход в систему» ", поля для ввода с наименованиями "Логин (ИНН)" и "Пароль", кнопка войти.
Шаг №2	Ввести инн и пароль с существующими данными в таблице, где пароль и инн будут соответствовать друг другу, нажать на кнопку "Войти" 
Поведение	Позитивное
Статус	Пройден

Таблица 10 - Тест-кейс №2

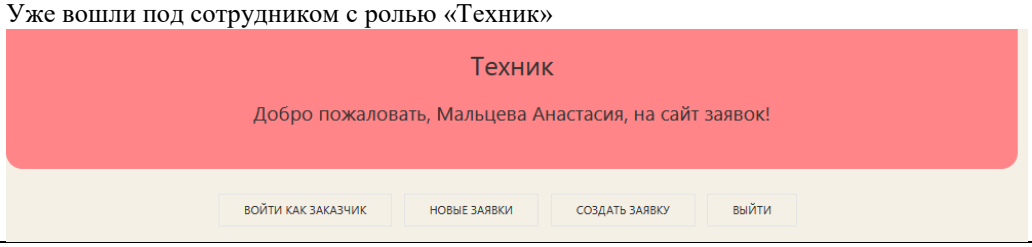
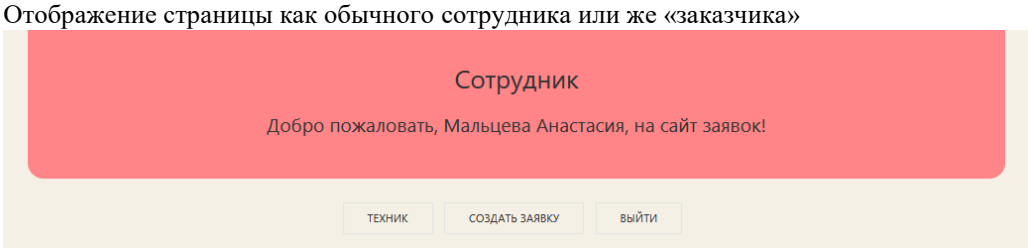
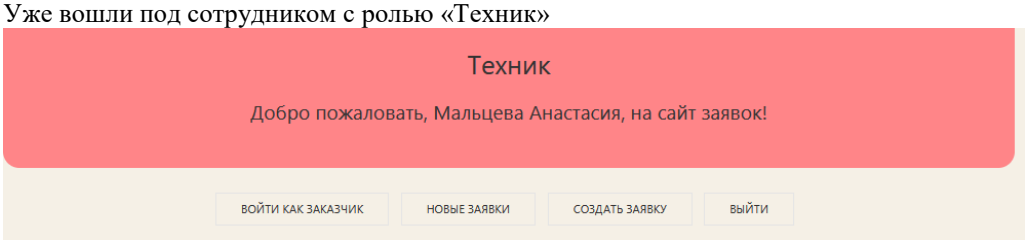
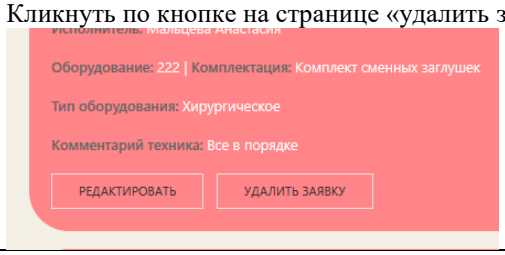
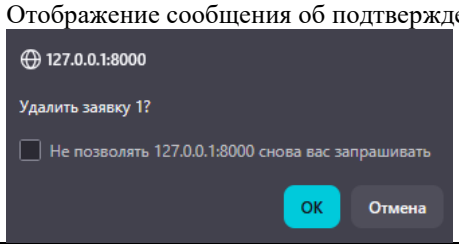
Номер	2
Название	Тестирование навигации из техника на заказчика
Предусловие	Уже вошли под сотрудником с ролью «Техник» 
Шаг №1	Кликнуть по кнопке на странице «войти как заказчик»
Результат шага №1	Отображение страницы как обычного сотрудника или же «заказчика» 
Поведение	Позитивное
Статус	Пройден

Таблица 11 - Тест-кейс №3

Номер	3
Название	Тестирование удаления заявки
Предусловие	Уже вошли под сотрудником с ролью «Техник» 
Шаг №1	Кликнуть по кнопке на странице «удалить заявку» 
Результат шага №1	Отображение сообщения об подтверждении удаления 
Шаг №2	Подтвердить удаление

Продолжение таблицы 11

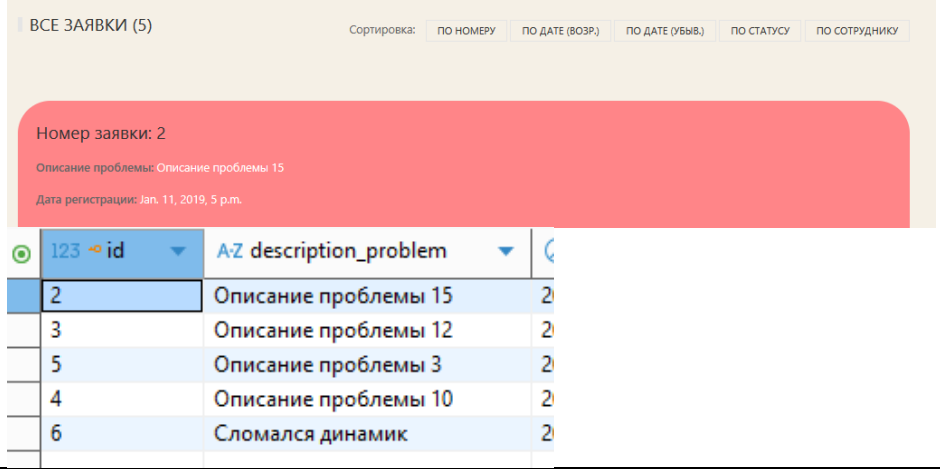
Результат шага №2	Заявки под порядковым номером «1» больше не существует в базе данных и на странице 
Поведение	Позитивное
Статус	Пройден

Таблица 12 - Тест-кейс №4

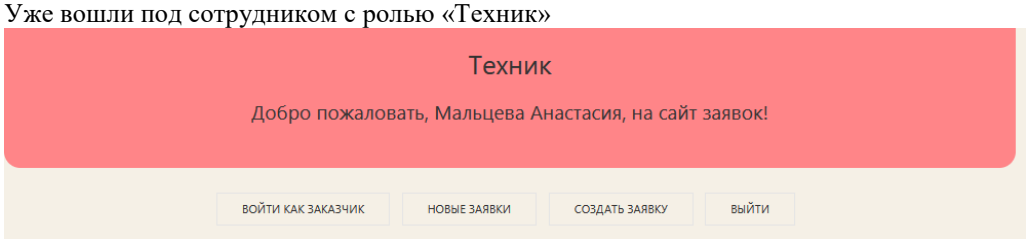
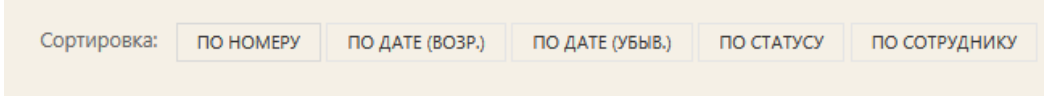
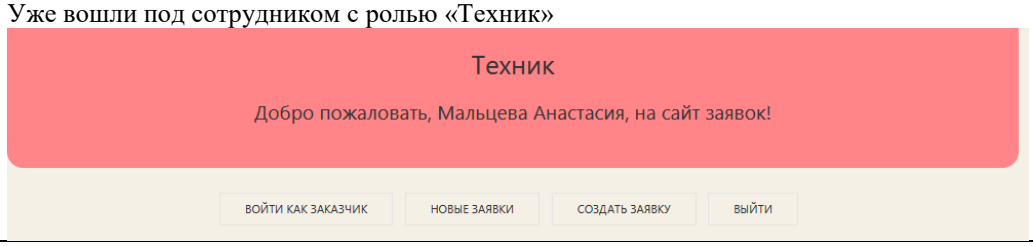
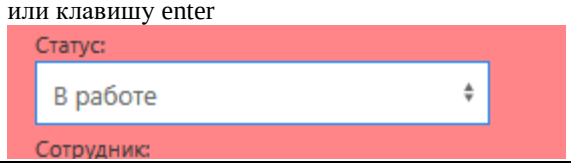
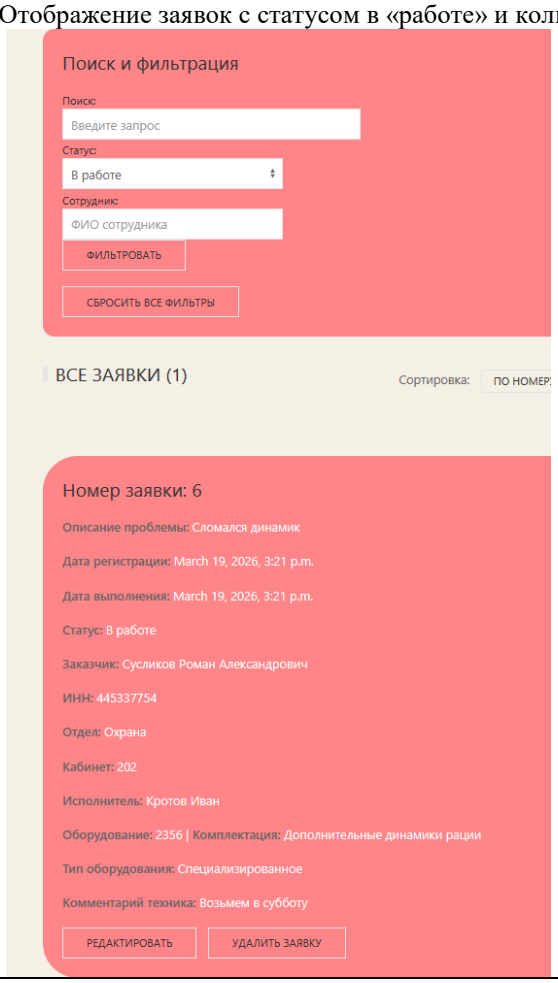
Номер	4
Название	Тестирование сортировки по убыванию на странице техника
Предусловие	Уже вошли под сотрудником с ролью «Техник» 
Шаг №1	Кликнуть по кнопке на странице «По дате (убыв.)» 
Результат шага №1	Отображение заявок от последней к самой первой
Поведение	Позитивное
Статус	Пройден

Таблица 13 - Тест-кейс №5

Номер	5
Название	Тестирование фильтрации по статусу
Предусловие	Уже вошли под сотрудником с ролью «Техник» 

Продолжение таблицы 13

Шаг №1	Выбрать в поле «статус» один из статусов, к примеру «В работе» и нажать на «фильтровать» или клавишу enter 
Результат шага №1	Отображение заявок с статусом в «работе» и количеству таких заявок 
Поведение	Позитивное
Статус	Пройден

4.2 Проведение модульного тестирования

Проведение модульного тестирования необходимо для верификации корректности работы отдельных компонентов программного средства и выявления ошибок логики на ранних этапах. Основной целью данного этапа является подтверждение того, что функции приложения выполняют заложенную логику без сбоев.

Для автоматизации процесса тестирования использовались встроенные средства фреймворка Django на языке программирования Python, соответствующие выбранным системным характеристикам проекта. В качестве инструментальной базы применялись классы

TestCase и клиент тестирования Django, позволяющие эмулировать HTTP-запросы и проверять ответы сервера, статусы моделей базы данных и корректность перенаправлений.

Такой подход позволил подтвердить работоспособность критических узлов системы, включая механизмы аутентификации, применения фильтров и сортировки, обеспечивая надежность и стабильность разработанного программного модуля перед его внедрением

Листинг 11 – Тестирование входа на сайт через логин и пароль

```
#Тест входа в систему
class TestLogin(TestCase):
    def setUp(self):
        self.role = Role.objects.create(role_name='Сотрудник')
        self.department = Department.objects.create(id=1, name='IT')
        self.office = Office.objects.create(id=101)
        self.employee = Employee.objects.create(
            id='123456789012',
            inn=123456789012,
            number_phone='+79001234567',
            first_name='Иванов',
            name='Иван',
            second_name='Иванович',
            position=self.role,
            department=self.department,
            office=self.office,
            password='test123'
        )
        self.client = Client()
    #Проверка входа
    def test_login_success(self):
        response = self.client.post(reverse('login_view'), {
            'username': '123456789012',
            'password': 'test123'
        })
        self.assertEqual(response.status_code, 302)
        self.assertEqual(self.client.session['user_inn'],
'123456789012')
```

```
PS C:\Users\Снежик\Desktop\equipment\equip> python manage.py test account_equipment.tests.TestLogin
Found 1 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
.
----------------------------------------------------------------------
Ran 1 test in 0.013s

OK
Destroying test database for alias 'default'...
```

Рисунок 4.1 – Успешный тест входа на сайт

Листинг 12 – Тестирование кнопки навигации

```
#Переход по навигации
class TestNavigation(TestCase):
    def setUp(self):
        self.role = Role.objects.create(role_name='Техник')
        self.department = Department.objects.create(id=1, name='IT')
        self.office = Office.objects.create(id=101)
        self.technician = Employee.objects.create(
            id='123456789012',
            inn=987654321098,
            number_phone='+79007654321',
            first_name='Петров',
            name='Петр',
            second_name='Петрович',
            position=self.role,
            department=self.department,
            office=self.office,
            password='tech123'
        )
        self.client = Client()
        self.client.post(reverse('login_view'), {
            'username': '987654321098',
            'password': 'tech123'
        })

    #Переход на заказчика через роль техника
    def test_navigation_to_customer(self):
        response = self.client.get(reverse('show_customer'))
        self.assertEqual(response.status_code, 200)
```

```
PS C:\Users\Снежик\Desktop\equipment\equip> python manage.py test account_equipment.tests.TestNavigation
Found 1 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
.
----------------------------------------------------------------------
Ran 1 test in 0.020s

OK
Destroying test database for alias 'default'...
```

Рисунок 4.2 – Успешный переход по кнопке навигации

Листинг 13 – Тестирование кнопки удаления

```
#Тестирование кнопки удаления
class TestDeleteButton(TestCase):
    def setUp(self):
        self.role = Role.objects.create(role_name='Техник')
        self.department = Department.objects.create(id=1, name='IT')
        self.office = Office.objects.create(id=101)
        self.technician = Employee.objects.create(
            id='123456789012',
            inn=9876543210,
```

Продолжение листинга 13

```
            number_phone='+79007654321',
            first_name='Петров',
            name='Петр',
            second_name='Петрович',
            position=self.role,
            department=self.department,
            office=self.office,
            password='tech123'
        )
        # Создание оборудования с типом
        self.equipment_type   =   EquipmentType.objects.create(id=1,
name='Компьютер')
        self.equipment = Equipment.objects.create(
            id=1001,
            set='Комплект 1',
            pin_office=self.office,
            type=self.equipment_type
        )
        self.status     =     ApplicationStatus.objects.create(id=1,
name='Новая')
        # Заявка
        self.request = RequestFix.objects.create(
            id=1,
            description_problem='Тест',
            date_register=timezone.now(),
            date_done=timezone.now(),
            comment_technician='Тест',
            customer=self.technician,
            given_technician=self.technician,
            equipment=self.equipment,
            status_done_request=self.status
        )
        self.client = Client()
        self.client.post(reverse('login_view'), {
            'username': '9876543210',
            'password': 'tech123'
        })
    #Тестирование удаление заявки
    def test_delete_request(self):
        self.assertEqual(RequestFix.objects.count(), 1)
        response    =    self.client.post(reverse('delete_request',
args=[1]))
        self.assertEqual(response.status_code, 302)
        self.assertEqual(RequestFix.objects.count(), 0)
```

```
PS C:\Users\Снежик\Desktop\equipment\equip> python manage.py test account_equipment.tests.TestDeleteButton
Found 1 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
.
----------------------------------------------------------------------
Ran 1 test in 0.018s

OK
Destroying test database for alias 'default'...
```

Рисунок 4.3 – Успешное удаление записи

Листинг 14 – Тестирование сортировки по убыванию на странице техника

```
#Тест сортировки по убыванию на странице техника
class TestSorting(TestCase):
    def setUp(self):
        self.role = Role.objects.create(role_name='Техник')
        self.department = Department.objects.create(id=1, name='IT')
        self.office = Office.objects.create(id=101)
        self.technician = Employee.objects.create(
            inn=9876543210,
            number_phone='+79007654321',
            first_name='Петров',
            name='Петр',
            second_name='Петрович',
            position=self.role,
            department=self.department,
            office=self.office,
            password='tech123'
        )
        self.equipment_type = EquipmentType.objects.create(id=1,
name='Компьютер')
        self.equipment = Equipment.objects.create(
            id=1001,
            set='Комплект 1',
            pin_office=self.office,
            type=self.equipment_type
        )
        self.status = ApplicationStatus.objects.create(id=1,
name='Новая')
        RequestFix.objects.create(
            id=1,
            description_problem='Тест 1',
            date_register=timezone.now(),
            date_done=timezone.now(),
            comment_technician='Тест',
            customer=self.technician,
            given_technician=self.technician,
            equipment=self.equipment,
            status_done_request=self.status
        )
        RequestFix.objects.create(
            id=2,
            description_problem='Тест 2',
```

Продолжение листинга 14

```
            date_register=timezone.now(),
            date_done=timezone.now(),
            comment_technician='Тест',
            customer=self.technician,
            given_technician=self.technician,
            equipment=self.equipment,
            status_done_request=self.status
        )
        self.client = Client()
        self.client.post(reverse('login_view'), {
            'username': '9876543210',
            'password': 'tech123'
        })
    def test_sort_by_date_desc(self):
        response    =    self.client.get(reverse('show_technician'),
{'sort': 'date_register_desc'})
        self.assertEqual(response.status_code, 200)
        self.assertContains(response, 'Тест 1')
        self.assertContains(response, 'Тест 2')
```

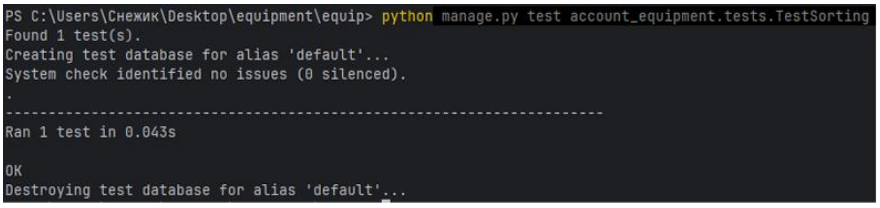


Рисунок 4.4 – Успешная сортировка по убыванию

Листинг 15 – Фильтрация по статусу

```
#Тест фильтрации по статусу
class TestFiltering(TestCase):
    def setUp(self):
        self.role = Role.objects.create(role_name='Техник')
        self.department = Department.objects.create(id=1, name='IT')
        self.office = Office.objects.create(id=101)
        self.technician = Employee.objects.create(
            inn=9876543210,
            number_phone='+79007654321',
            first_name='Петров',
            name='Петр',
            second_name='Петрович',
            position=self.role,
            department=self.department,
            office=self.office,
            password='tech123'
        )
        self.equipment_type   =   EquipmentType.objects.create(id=1,
name='Компьютер')
```

Продолжение листинга 15

```
        self.equipment = Equipment.objects.create(
            id=1001,
            set='Комплект 1',
            pin_office=self.office,
            type=self.equipment_type
        )
        self.status_new   =   ApplicationStatus.objects.create(id=1,
name='Новая')
        self.status_done   =   ApplicationStatus.objects.create(id=2,
name='Выполнена')
        RequestFix.objects.create(
            id=1,
            description_problem='Проблема с мышью',
            date_register=timezone.now(),
            date_done=timezone.now(),
            comment_technician='Тест',
            customer=self.technician,
            given_technician=self.technician,
            equipment=self.equipment,
            status_done_request=self.status_new
        )
        RequestFix.objects.create(
            id=2,
            description_problem='Проблема с клавиатурой',
            date_register=timezone.now(),
            date_done=timezone.now(),
            comment_technician='Готово',
            customer=self.technician,
            given_technician=self.technician,
            equipment=self.equipment,
            status_done_request=self.status_done
        )
        self.client = Client()
        self.client.post(reverse('login_view'), {
            'username': '9876543210',
            'password': 'tech123'
        })
    def test_filter_by_status(self):
        response    =    self.client.get(reverse('show_technician'),
{'status': '1'})
        self.assertEqual(response.status_code, 200)
        self.assertContains(response, 'Проблема с мышью')
        self.assertNotContains(response, 'Проблема с клавиатурой')
```

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы по МДК 01.01 «Разработка модулей программного обеспечения для компьютерных систем» была успешно достигнута цель курсовой работы — разработка и поддержка программного обеспечения для учета заявок на исправление технических неисправностей техники в ГБУЗ РХ «АМКБ».

Были решены следующие задачи:

- Спроектирована функциональная архитектура программного средства;
- Спроектирована структура базы данных;
- Реализован функционал архитектуры программного средства;
- Проведено модульное тестирование.

Достигнутые результаты полностью соответствуют поставленной цели — созданию и поддержке программного учета заявок по неисправностям техники в ГБУЗ РХ «АМКБ». Разработанное приложение позволяет автоматизировать ключевые процессы, сократить время обработки заявок и повысить прозрачность взаимодействия между подразделениями ГБУЗ РХ «АМКБ».

Разработанное программное решение обладает необходимым запасом масштабируемости и модульности, что позволит в будущем интегрировать его с другими информационными системами больницы или адаптировать под изменения в нормативно-правовой базе без существенной переработки.

Таким образом, курсовая работа продемонстрировала не только теоретическую подготовку, но и практические навыки в области проектирования, разработки и тестирования программного обеспечения. Полученные результаты могут служить основой для дальнейшего развития системы, получения прямых уведомлений о поступивших заявках, учета затрат на ремонт и внедрения расширенной аналитики для прогнозирования износа оборудования.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Соммервилл И. Проектирование программного обеспечения: пер. с англ. — 10-е изд. — М.: Вильямс, 2022. — 784 с.
2. Буч Г., Рамбо Дж., Якобсон И. Язык UML. Руководство пользователя: пер. с англ. — 2-е изд. — М.: ДМК Пресс, 2020. — 496 с.
3. Коннелли Дж. Реляционные базы данных: теория и практика: пер. с англ. — М.: Вильямс, 2019. — 416 с.
4. Глушаков С.В., Ломотько Д.В. Проектирование информационных систем. — Харьков: Фолио, 2004. — 432 с.
5. Маклаков С.В. BPwin и ERwin. CASE-средства разработки информационных систем. — М.: Диалог-МИФИ, 2003. — 320 с.
6. Орлов С.А., Цилькер Б.Я. Технологии разработки программного обеспечения. — 3-е изд., испр. — СПб.: Питер, 2020. — 608 с.
7. Маклаков С.В. Моделирование бизнес-процессов с использованием IDEF0 // Компьютерные инструменты в образовании. — 2002. — № 2. — С. 45–58.
8. Шабанов Д.А., Симонов А.А. Подходы к проектированию архитектуры информационных систем // Вестник ЮУрГУ. Серия «Вычислительная математика и информатика». — 2018. — Т. 7, № 3. — С. 82–95.
9. Davis A.M. Software Requirements: Objects, Functions, and States // Communications of the ACM. — 1993. — Vol. 36, № 3. — P. 68–83.
10. Yourdon E. Modern Structured Analysis: A Retrospective // IEEE Software. — 1988. — Vol. 5, № 6. — P. 11–18.
11. Django Documentation [Электронный ресурс]. — URL: https://docs.djangoproject.com/ (дата обращения: 23.01.2026).
12. PostgreSQL Documentation [Электронный ресурс]. — URL: https://www.postgresql.org/docs/ (дата обращения: 06.03.2026).
13. UML Specification. Version 2.5.1 / Object Management Group (OMG) [Электронный ресурс]. — URL: https://www.omg.org/spec/UML/ (дата обращения: 19.12.2025).
14. IDEF0 Standard (FIPS PUB 183) [Электронный ресурс]. — URL: https://www.idef.com/idef0/ (дата обращения: 10.04.2026).

ПРИЛОЖЕНИЯ

Ссылка на систему контроля версий

GIT - https://github.com/SnezhiK000/Accounting_equipment_requests